

Assignment 4

Go to login page.

Steps to Test for Brute Force Attack:

1. Go to login page.
2. Use Burp Suite to Intercept and Modify the Request:
3. Set username as admin
4. configure the Payload Type as Simple List with a dictionary of potential passwords
5. Start Attack in Burp Suite.
6. Monitor the Response Status Codes and Response Length for each request.
7. the server will respond with a 200 OK if login successful
8. If the status code is different (e.g., 403 Forbidden, 404 Not Found), this may indicate the login attempt failed.

Mitigation of Brute Force Attacks:

To prevent brute-force attacks, we have implemented a **lockout mechanism**. Here's how it works:

- **Account Lockout:** After **5 consecutive failed login attempts**, the system will lock the account for **15 minutes**.
- **Error Message:** The error message displayed after multiple failed attempts informs the user that they must wait before trying again.

Code for Brute Force Mitigation:

Here is the relevant PHP code in place to handle brute-force protection using a **lockout time** and **failed attempts tracking**:

```
$error = "";
$max_attempts = 5; // Maximum failed attempts before blocking
login
$lockout_time = 900; // Lock user out for 15 minutes after max
attempts

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $username = trim($_POST['username']);
    $password = $_POST['password'];

    // Input validation: Ensure username and password are not
empty
    if (empty($username) || empty($password)) {
        $error = "Username and password are required!";
    } else {
        // Prevent brute-force by tracking failed login attempts
        $stmt = $conn->prepare("SELECT id, username, role,
password FROM users WHERE username = ?");
        $stmt->bind_param("s", $username);
        $stmt->execute();
        $result = $stmt->get_result();

        if ($result->num_rows > 0) {
            $user = $result->fetch_assoc();
            $user_id = $user['id'];

            // Enforce lockout policy
            if ($conn->query("SHOW COLUMNS FROM users LIKE
'failed_attempts'")->num_rows) {
                $stmt = $conn->prepare("SELECT failed_attempts,
last_attempt FROM users WHERE username = ?");
                $stmt->bind_param("s", $username);
                $stmt->execute();
                $extra_result = $stmt->get_result();
                $extra_data = $extra_result->fetch_assoc();

                $user['failed_attempts'] =
$extra_data['failed_attempts'];
                $user['last_attempt'] =
$extra_data['last_attempt'];

                if ($user['failed_attempts'] >= $max_attempts &&
(time() - strtotime($user['last_attempt'])) < $lockout_time) {
                    logSecurityEvent($conn, "Account Locked -
```

```

Too many failed login attempts", $user_id);
        $error = "Too many failed login attempts.
Please try again after 15 minutes.";
    }
}

    if (empty($error)) {
        if (password_verify($password,
$user['password'])) {
            // Secure session management
            session_regenerate_id(true);

            $SESSION['user id'] = $user id;
            $_SESSION['username'] = $user['username'];

            // Strict role validation
            $allowed_roles = ['admin', 'user'];
            $_SESSION['role'] = in_array($user['role'],
$allowed_roles) ? $user['role'] : 'user';

            // Log successful login
            logSecurityEvent($conn, "Successful Login",
$user_id);

            // Redirect based on role
            header("Location: " . ($_SESSION['role'] ===
'admin' ? "admin_dashboard.php" : "user_dashboard.php"));
            exit();
        } else {
            // Update failed login attempts
            if (isset($user['failed_attempts'])) {
                $update_stmt = $conn->prepare("UPDATE
users SET failed_attempts = failed_attempts + 1, last_attempt =
NOW() WHERE id = ?");
                $update_stmt->bind_param("i", $user_id);
                $update_stmt->execute();
            }

            // Log failed login attempt
            logSecurityEvent($conn, "Failed Login -
Incorrect password", $user_id);
            $error = "Invalid username or password!";
        }
    }
} else {
    // Log failed login attempt for unknown username
    logSecurityEvent($conn, "Failed Login - Unknown

```

```
username: $username", NULL);  
    $error = "Invalid username or password!";  
}  
}  
}
```